

**PREDICTING CYBERSECURITY THREAT SEVERITY
USING MACHINE LEARNING:
A CRITICAL EVALUATION OF SYNTHETIC DATASET
VALIDITY IN BUSINESS ANALYTICS**

Sanjai Balaji
(40478904)

Word Count: 2660

Technical Report, Logbook and Reflective Discussion
submitted in part fulfilment of the degree of
Master of Science in Business Analytics

2026

Queen's Business School
Queen's University Belfast

Declaration

This is to certify that:

- i. The portfolio comprises only my original work;
- ii. AI technologies (e.g. ChatGPT) have not been used in the writing of the portfolio dissertation;
- iii. No portion of the work referred to in the dissertation has been submitted in support of an application for another degree or qualification of this or any other university.

Sanjai Balaji
(Candidate's Signature)

Sanjai Balaji
(Printed Name)

11-09-2026

Table of Contents

Declaration	2
Contents	3
1. Technical Introduction	4
2. Solution Architecture	4
3. Data Acquisition and Pre-processing	5
4. Exploration and Visualisation	6
5. Analytics Implementation	7
6. Technical Discussion	8
7. Conclusions and Further Development.....	9
8. Logbook	9
9. Reflective Discussion.....	13
Appendix 1 — R Analysis Script.....	15
Appendix 2 — Session Information.....	30
Appendix 3 — Screenshots.....	31

1. Technical Introduction and Background

This report documents the technical implementation, the design decisions taken, and the alternatives rejected. It does not restate the research report methodology; its focus is the reasoning behind technical choices.

The solution originated as a regression pipeline predicting threat severity. When modelling returned no performance, it was extended with a diagnostic layer establishing whether the failure lay in the pipeline or the data.

The literature informing the modelling component is well established, with Dasari and Kaluri (2024) and Mehmood et al. (2023) implementing comparative ensemble pipelines that informed this design. Less precedent exists for the diagnostic component.

2. Solution Architecture and Technical Design

The solution is a single reproducible R script of approximately 300 lines in thirteen sequential blocks, writing thirteen tables, ten figures and a session information file.

2.1 Language selection

Python was used for preliminary exploratory analysis at proposal stage using pandas, matplotlib and seaborn. Three considerations prompted migration to R for the main analysis.

First, the diagnostic component depends on formal hypothesis testing — Kolmogorov-Smirnov, chi-square, ANOVA with effect sizes and regression diagnostics — which R provides natively with richer output than Python equivalents.

Second, R's regression summary output is substantially richer, which mattered for the internal consistency test where coefficient significance carried the argument.

Third, a practical constraint intervened. Python encountered a `MemoryError` during one-hot encoding, attempting to allocate a 50,000 by 49,776 boolean array. R's factor representation avoids this entirely, as documented in logbook entry 3.

Power BI was specified at proposal stage but not implemented. Once no predictive relationship existed, the figures required became statistical diagnostics Power BI cannot produce without external calculation.

2.2 Package selection

Package	Purpose	Rationale
stringr	JSON field extraction	Lookbehind regex support for named-key parsing
dplyr / tidyr	Data manipulation and reshaping	Pipeline syntax; pivot_longer for faceted plotting
ggplot2	All visualisation	Consistent theming; reproducible from script
randomForest	Ensemble regression and classification	Native permutation importance via importance()
rpart	Decision tree	Diagnostic baseline: reveals when no split is worthwhile
xgboost	Gradient boosting	Highest-flexibility model; tests the overfitting hypothesis
caret	Stratified partitioning	createDataPartition preserves target distribution
corrplot	Correlation heatmap	Fixed colour limits render near-zero values honestly

3. Data Acquisition, Storage and Pre-processing

3.1 Acquisition and storage

The dataset was obtained as a single CSV of 50,000 records, stored on a password-protected device with an encrypted backup. No database layer was implemented, since the file sits within memory and is read once.

3.2 The nested JSON problem

Initial inspection revealed the principal technical challenge: seven of twenty-two columns contained nested structures rather than scalar values, four holding numeric fields of analytical interest.

```
{'existing_controls_score': 0.8353696, 'control_categories': {'preventive': 0.8131574, 'detective': 0.7194482}}
```

Two strategies were considered. Full JSON parsing via jsonlite fails because the fields use Python dictionary syntax with single quotes, which is not valid JSON. Regex extraction with positive lookbehind was adopted instead.

The approach adopted uses positive lookbehind regular expressions to extract named numeric fields directly, implemented as a reusable helper:

```
ext_num <- function(col, key) as.numeric(str_extract(col, paste0("(?<=", key, ":[0-9.]+)")))
```

This is vectorised in a single call but brittle: it depends on exact key naming and returns NA silently if the schema varies. Extraction was validated across all 50,000 records.

3.3 Cleaning and preparation

No imputation was required, as the dataset contained no missing values or duplicates.

One variable was constant across all records and excluded for zero variance.

4. Data Exploration and Visualisation

Ten figures were produced in ggplot2 with a shared theme applied globally via theme_set, guaranteeing typographic consistency with the written reports.

Three visualisation decisions were made specifically to represent the findings honestly rather than attractively, since each concerns how absence of structure is rendered.

In the histograms a reference line marks the count expected under uniformity. In the heatmap the colour scale is fixed at the full theoretical range. In the importance plot the axis extends below zero, since negative values are a finding.

Figures were saved programmatically via a save_fig wrapper rather than exported manually, ensuring that dimensions, resolution and background are identical across all outputs and that re-running the script overwrites cleanly.

5. Analytics Implementation

5.1 Modelling pipeline

The modelling component evaluates four algorithms across four targets under five-fold cross-validation, spanning functional flexibility rather than maximising performance.

Cross-validation was implemented manually rather than through `caret::train`, to retain explicit control over fold assignment and to compute a per-fold naive baseline alongside each model.

A naive baseline predicting the training-fold mean was computed alongside every model on each held-out fold, giving negative R-squared values a concrete reference point.

5.2 A target leakage error and its correction

The most significant technical error occurred here. In the initial configuration the target variable remained within the predictor set, producing an R-squared of 1.000 that was investigated rather than reported.

The result was an R-squared of 1.000, investigated rather than reported. The correction removes the target within the modelling function rather than relying on caller-supplied lists.

```
preds <- setdiff(features, target_name)
```

Re-running produced -0.0035 , -0.0001 and -0.0144 , consistent with the other three targets.

5.3 Diagnostic pipeline

Five diagnostic procedures were implemented: distribution testing via Kolmogorov–Smirnov and chi-square, correlation and association screening, internal consistency regression, permutation testing and a learning curve.

Correlation and association screening computes the full pairwise matrix alongside one-way ANOVA with eta-squared for each categorical predictor, since a numeric-only matrix would leave categorical variables untested.

The permutation test refits the model ten times with randomly shuffled training targets, shuffling only the training set so the comparison isolates whether anything transferable was learned.

5.4 Memory management and reproducibility

Tree models were fitted on a fixed random subsample of 10,000 records with capped depth, a memory constraint documented in logbook entry 3. Statistical tests use the complete dataset throughout.

A single seed governs all sampling, partitioning and model fitting, set once at the top of the script, so every reported figure regenerates identically from a clean environment.

6. Technical Discussion

The solution differs from the literature in objective rather than construction: the modelling component follows established practice, while the diagnostic component has no direct equivalent.

This produces one architectural inversion: in a conventional pipeline validation follows modelling, whereas here the diagnostic layer became the primary deliverable and modelling the evidence supporting it.

The closest precedents are Moustafa and Slay (2016), who statistically evaluate their own dataset, and Tavallaei et al. (2009), who analyse KDD Cup 99 — but neither presents a reusable implementation.

Two literature techniques were excluded deliberately: SMOTE addresses imbalance this dataset does not exhibit, and LASSO addresses dimensionality seventeen predictors do not present.

7. Technical Conclusions and Further Development

The solution meets its revised objective, executing end to end in roughly eight minutes and producing every table and figure reported.

7.1 Limitations of the implementation

Four limitations apply: a 10,000-record subsample for tree models owing to memory; hyperparameter optimisation limited to defaults; regex extraction brittle to schema variation; and one learning-curve fit per sample size.

7.2 Recommendations for further development

Three developments would strengthen the solution: packaging the diagnostic layer as a reusable function returning a structured report; averaging five repetitions per learning-curve point; and adding schema validation to the extraction stage.

8. Logbook

Six entries recording work undertaken and technical decisions taken across the project.

Entry 1 — approx. 8 June 2026

Period covered: early June

Reviewed the dissertation handbook and marking rubric, and identified cybersecurity threat prediction as the topic area on the basis of business relevance and personal interest. Searched public repositories for a suitable dataset and selected a 50,000-record cybersecurity threat scenario file containing 22 columns including a continuous `threat_intelligence_score`.

Technical decision: selected regression over classification as the modelling paradigm. The target variable is continuous, and discretising it into severity bands would have discarded granular information. This was recorded in the proposal with justification.

Assessed the dataset on conventional quality criteria: 50,000 records, no missing values, no duplicates, balanced categorical distributions. Concluded it was suitable. In retrospect

this assessment was necessary but not sufficient, and its limitations became the subject of the project.

Entry 2 — approx. 9 July 2026

Period covered: mid-June to early July

Completed preliminary analysis in Python using pandas, matplotlib and seaborn, producing five figures for the proposal: target distribution, threat type and attack vector counts, a boxplot by threat type, and a correlation heatmap.

Technical observation: the correlation heatmap returned values between -0.01 and 0.01 for every pair, and `historical_incident_count` showed zero variance and rendered blank. At this stage these were interpreted as a modelling challenge to be addressed through ensemble methods rather than as evidence of a data defect. This interpretation was recorded in the proposal as the first identified project risk.

Finalised and submitted the proposal and ethics approval form. In hindsight, the correlation result warranted investigation rather than mitigation planning, and recognising that distinction earlier would have saved several weeks.

Entry 3 — approx. 20 July 2026

Period covered: early July to late July

Attempted the first full modelling run. In Python, `pd.get_dummies` raised a `MemoryError` attempting to allocate 2.32 GiB for a $50,000 \times 49,776$ array. Diagnosis established that `assets_affected` contains near-unique values per record and was being one-hot encoded.

Technical decision: filtered categorical columns to those with fewer than 50 unique levels before encoding. This resolved the allocation failure and reduced the feature count to twenty.

First successful model fit returned R-squared of -0.021 on the test set — worse than predicting the mean. A second memory failure then occurred during random forest tree construction.

Technical decision: migrated the analysis to R. Factor representation avoids dense indicator matrices, and R's statistical testing and regression summary facilities were better suited to the diagnostic work that the negative result now clearly required.

Entry 4 — approx. 10 August 2026

Period covered: late July to early August

Systematic inspection of column structure in R revealed that seven columns contained serialised JSON rather than scalar values, concealing eleven additional numeric variables that had not been present in any prior analysis.

Technical decision: implemented extraction using positive lookbehind regular expressions via stringr rather than full JSON parsing with jsonlite. The strings use single quotes, so jsonlite would have required pre-processing every record; regex extraction is vectorised and substantially faster.

Re-ran the correlation analysis with the expanded 16-variable set. All 120 pairwise correlations remained below 0.0112. Critically, `existing_controls_score` showed a correlation of -0.003 with preventive and -0.007 with detective — its own stated sub-components. Formal regression returned R-squared of 0.000058 with no significant coefficients.

This was the decisive finding of the project. A composite measure cannot be uncorrelated with its own inputs unless the fields were generated independently. The interpretation shifted at this point from weak signal to internal incoherence.

Entry 5 — approx. 14 August 2026

Period covered: early August to mid-August

Reported the findings to my supervisor with supporting evidence and three proposed options: replace the dataset, reframe as classification, or document the negative result and pivot to dataset evaluation. Following supervisory guidance, given the remaining timeline, the third option was adopted and the title amended accordingly.

Built the full diagnostic battery: five-fold cross-validation across four algorithms and four targets, dual uniformity testing, categorical ANOVA with eta-squared, permutation importance, label-permutation testing, learning curve and classification re-framing.

Technical error identified and corrected: the initial cross-validation function returned R-squared of 1.000 for critical_assets. Investigation established that the target was present in the predictor set. Corrected by computing predictors dynamically with setdiff, after which the result fell to -0.0144 , consistent with the other targets. This was the most instructive error of the project — a perfect score should trigger investigation, not satisfaction.

Technical decision: capped tree parameters and subsampled to 10,000 records for modelling to remain within available memory, whilst retaining all 50,000 records for statistical testing.

Entry 6 — approx. 5 September 2026

Period covered: mid-August to early September

Consolidated the analysis into a single reproducible script of approximately 300 lines with a fixed seed, generating all thirteen tables and ten figures programmatically to a structured output directory.

Technical decision: did not implement the planned SHAP analysis. Explaining the predictions of a model with no predictive validity would produce interpretations of noise. This was recorded as a deliberate methodological judgement in both reports rather than presented as an omission.

Technical decision: fixed corplot colour limits at -1 to $+1$ rather than accepting the default observed-range scaling, which would have rendered correlations of 0.01 in saturated colour and visually overstated relationships that do not exist.

Expanded the literature review from five to twenty-one sources, adding work on synthetic data quality, dataset documentation and cybersecurity dataset critique. Developed the five-stage validation protocol as the project's methodological contribution, and completed both reports.

9. Reflective Discussion

9.1 What I Set Out to Do and What Happened

I began intending to build a predictive model and finished having built a diagnostic one. The distance between those outcomes is where my development took place.

The defining moment came when a random forest returned a negative R-squared. I assumed a coding error and rebuilt the pipeline before accepting the result — debugging code when I should have interrogated data.

9.2 Technical Skills Developed

The most substantial development was in diagnostic reasoning — working backwards from an unexpected result and designing tests that discriminate between candidate explanations.

I also became genuinely bilingual across Python and R, having implemented comparable pipelines in both and encountered the practical consequences of their differing treatment of categorical data.

Reproducibility became a working practice rather than a principle. A single fixed seed governing all sampling means every figure regenerates identically, which mattered when results had to be defended.

9.3 What I Found Difficult

The hardest part was the period between discovering the models failed and establishing why. Emailing my supervisor with structured evidence and three options, rather than concealing the problem, proved correct.

The target leakage error was chastening differently. An R-squared of 1.000 produced relief before suspicion. I now treat any near-perfect metric as a defect indicator until demonstrated otherwise.

Communicating a negative finding was also difficult. Reporting that the dataset might not support the project felt like reporting my own failure, until the evidence made clear it was not.

9.4 What I Would Do Differently

I would validate before committing. The correlation matrix that proved decisive was computed at proposal stage but recorded as a risk rather than investigated.

I would investigate provenance before downloading. I selected the dataset on size and completeness and never recorded the source URL or licence.

I would also design the learning curve more carefully; a single fit per sample size produced my noisiest figure.

9.5 Application to Professional Practice

The transferable lesson concerns apparent quality versus analytical validity. Commercially, datasets are typically internal and undocumented, where these failure modes are common.

The validation protocol is something I intend to use professionally; its stages are cheap enough to run routinely.

Finally, this project taught me to report findings I did not want to find — the conversation an analyst has when data does not support a hoped-for decision.

Appendices

Appendix 1 — Complete R Analysis Script

```
# =====  
  
# MSc Business Analytics Dissertation - Queen's University Belfast  
  
# Predicting Cybersecurity Threat Severity Using Machine Learning:  
  
# A Critical Evaluation of Synthetic Dataset Validity in Business Analytics  
  
#  
  
# Author: Sanjai Balaji (40478904)  
  
#  
  
# PURPOSE  
  
# Reproduces every table (T01-T12) and figure (F01-F09) used in the  
  
# Research Report and Technical Report. Run top to bottom in R/RStudio.  
  
# All outputs are written to ./outputs/tables and ./outputs/figures.  
  
#  
  
# STRUCTURE (matches CRISP-DM phases referenced in the Methodology chapter)  
  
# 0 Setup  
  
# 1 Data Understanding - load, inventory, JSON extraction  
  
# 2 Data Quality - missing, duplicates, constants, invalid values  
  
# 3 Descriptive Stats - T03, F01  
  
# 4 Distribution Tests - KS + chi-square uniformity, T04, F02  
  
# 5 Correlation - T05, F03  
  
# 6 Internal Consistency- T06, F04  
  
# 7 Categorical Assoc. - T07, F05  
  
# 8 Predictive Modelling- 5-fold CV, 4 targets x 4 models, T08, F06  
  
# 9 Feature Importance - T09, F07  
  
# 10 Permutation Test - T10, F08  
  
# 11 Learning Curve - T11, F09  
  
# 12 Classification Test - T12  
  
# 13 Session Info - reproducibility  
  
#  
  
# NOTE ON MEMORY
```

```

# Random Forest / XGBoost are fitted on a fixed random sample of 10,000 rows
# (seed 40478904) with capped tree depth to keep memory under ~2 GB. This is
# documented as a design decision in the Technical Report. Statistical tests
# (sections 3-7) use all 50,000 rows.
# =====

# ---- 0. SETUP -----

pkgs <- c("stringr", "dplyr", "tidyr", "ggplot2", "randomForest", "rpart", "xgboost",
         "corrplot", "scales")

to_install <- pkgs[!pkgs %in% installed.packages()[, "Package"]]
if (length(to_install)) install.packages(to_install)
invisible(lapply(pkgs, library, character.only = TRUE))

set.seed(40478904)

dir.create("outputs",          showWarnings = FALSE)
dir.create("outputs/tables",   showWarnings = FALSE)
dir.create("outputs/figures",  showWarnings = FALSE)

# >>> EDIT THIS PATH TO YOUR FILE <<<

DATA_PATH <- "D:/Sanjai/Dissertation
Project/Dataset/cybersecurity_risk_dataset_50000.csv"

# Consistent figure style (Times New Roman to match the dissertation body text)

theme_diss <- theme_minimal(base_size = 11, base_family = "serif") +

  theme(plot.title = element_text(face = "bold", size = 12),

        panel.grid.minor = element_blank(),

        legend.position = "bottom")

theme_set(theme_diss)

save_fig <- function(p, name, w = 6.5, h = 4.2) {

  ggsave(file.path("outputs/figures", paste0(name, ".png")), p,

```

```

        width = w, height = h, dpi = 300, bg = "white")
    }

    save_tab <- function(x, name) {

        write.csv(x, file.path("outputs/tables", paste0(name, ".csv")), row.names = FALSE)

    }

    r2 <- function(actual, pred) 1 - sum((actual - pred)^2) / sum((actual - mean(actual))^2)

    rmse <- function(actual, pred) sqrt(mean((actual - pred)^2))

# ---- 1. DATA UNDERSTANDING -----

df_raw <- read.csv(DATA_PATH, stringsAsFactors = FALSE)

cat("Loaded:", nrow(df_raw), "rows x", ncol(df_raw), "columns\n")

# T01 - Column inventory: type, cardinality, missingness

inventory <- data.frame(

    column      = names(df_raw),

    type        = sapply(df_raw, function(x) class(x)[1]),

    n_unique    = sapply(df_raw, function(x) length(unique(x))),

    n_missing   = sapply(df_raw, function(x) sum(is.na(x) | x == "")),

    row.names   = NULL, stringsAsFactors = FALSE)

inventory$pct_unique <- round(100 * inventory$n_unique / nrow(df_raw), 1)

inventory$sis_nested_json <- grepl("^\\{\\^\\{\\[", sapply(df_raw, function(x)
as.character(x[1])))

save_tab(inventory, "T01_column_inventory")

print(inventory)

# Extract numeric fields embedded in JSON-like text columns

df <- df_raw

ext_num <- function(col, key) as.numeric(str_extract(col, paste0("(?<=", key, "': ) [0-
9.]+")))

df$hw_count      <- ext_num(df$asset_count,      "hardware")

df$sw_count      <- ext_num(df$asset_count,      "software")

df$critical_assets <- ext_num(df$asset_count,    "critical_assets")

```

```

df$revenue_per_hour <- ext_num(df$business_context, "revenue_per_hour")

df$revenue_annual <- ext_num(df$business_context, "revenue_annual")

df$data_value_per_gb <- ext_num(df$business_context, "data_value_per_gb")

df$industry_sector <- str_extract(df$business_context, "(?<='industry_sector': ')[A-Za-z ]+")

df$controls_score <- ext_num(df$existing_controls, "existing_controls_score")

df$preventive <- ext_num(df$existing_controls, "preventive")

df$detective <- ext_num(df$existing_controls, "detective")

df$prob_occurrence <- ext_num(df$prediction_results, "probability_of_occurrence")


cat("After JSON extraction:", ncol(df), "columns\n")


# Variable groups used throughout

NUM_VARS <- c("threat_intelligence_score", "hardware_risk_avg", "software_risk_avg",
              "network_segment_risk", "data_classification_risk",
              "avg_days_since_last_similar", "hw_count", "sw_count", "critical_assets",
              "revenue_per_hour", "revenue_annual", "data_value_per_gb",
              "controls_score", "preventive", "detective", "prob_occurrence")

CAT_VARS <- c("scenario_type", "threat_type", "attack_vector", "attack_complexity",
              "initial_access_method", "industry_sector")

for (v in CAT_VARS) df[[v]] <- as.factor(df[[v]])

TARGETS <-
c("threat_intelligence_score", "prob_occurrence", "controls_score", "critical_assets")


# ---- 2. DATA QUALITY -----

const_cols <- names(df)[sapply(df, function(x) is.numeric(x) && sd(x, na.rm = TRUE) ==
0)]

invalid_prob <- sum(df$prob_occurrence > 1, na.rm = TRUE)


# T02 - Data quality summary

dq <- data.frame(

  check = c("Total records",

            "Raw columns",

```

```

"Columns after JSON extraction",

"Duplicate rows",

"Numeric columns with missing values",

"Constant (zero-variance) numeric columns",

"High-cardinality text columns (>90% unique)",

"prob_occurrence values > 1.0 (invalid probability)",

"prob_occurrence maximum observed"),

result = c(nrow(df), ncol(df_raw), ncol(df),

sum(duplicated(df_raw)),

sum(sapply(df[NUM_VARS], function(x) any(is.na(x)))),

ifelse(length(const_cols) == 0, "none", paste(const_cols, collapse = ";

")),

paste(inventory$column[inventory$pct_unique > 90], collapse = "; "),

invalid_prob,

round(max(df$prob_occurrence, na.rm = TRUE), 3)),

stringsAsFactors = FALSE)

save_tab(dq, "T02_data_quality")

print(dq)


# ---- 3. DESCRIPTIVE STATISTICS -----

# T03

desc <- df %>%

select(all_of(NUM_VARS)) %>%

pivot_longer(everything(), names_to = "variable") %>%

group_by(variable) %>%

summarise(n = sum(!is.na(value)),

mean = mean(value, na.rm = TRUE), sd = sd(value, na.rm = TRUE),

min = min(value, na.rm = TRUE), q25 = quantile(value, .25, na.rm = TRUE),

median = median(value, na.rm = TRUE), q75 = quantile(value, .75, na.rm =

TRUE),

max = max(value, na.rm = TRUE), .groups = "drop") %>%

mutate(across(where(is.numeric), ~ round(.x, 4)))

```

```

save_tab(desc, "T03_descriptive_statistics")

print(desc, n = Inf)

# F01 - Faceted histograms of the six [0.5,1] bounded scores.

# Dashed line = expected count per bin under a perfectly uniform distribution.

score_vars <- c("threat_intelligence_score", "hardware_risk_avg", "software_risk_avg",
               "network_segment_risk", "data_classification_risk", "controls_score")

long_scores <- df %>% select(all_of(score_vars)) %>%

  pivot_longer(everything(), names_to = "variable", values_to = "value")

bins <- 25

exp_per_bin <- nrow(df) / bins

p1 <- ggplot(long_scores, aes(value)) +

  geom_histogram(bins = bins, fill = "steelblue", colour = "white", linewidth = .2) +

  geom_hline(yintercept = exp_per_bin, linetype = "dashed", colour = "firebrick") +

  facet_wrap(~ variable, ncol = 3) +

  labs(title = "Distribution of bounded score variables (n = 50,000)",

        subtitle = "Dashed line = expected count per bin under a uniform(0.5, 1.0)
distribution",

        x = NULL, y = "Count")

save_fig(p1, "F01_score_distributions", h = 4.8)

# ---- 4. DISTRIBUTION (UNIFORMITY) TESTS -----

# Two independent tests per variable:

#   (a) One-sample Kolmogorov-Smirnov vs U(a,b)   (note: ties warning is benign)

#   (b) Chi-square goodness-of-fit over 20 equal-width bins vs uniform expected

unif_test <- function(x, a, b, nb = 20) {

  x <- x[!is.na(x) & x >= a & x <= b]

  ks <- suppressWarnings(ks.test(x, "punif", a, b))

  obs <- table(cut(x, breaks = seq(a, b, length.out = nb + 1), include.lowest = TRUE))

  cs <- chisq.test(obs, p = rep(1/nb, nb))

  c(n = length(x), ks_D = unname(ks$statistic), ks_p = ks$p.value,

```

```

    chisq = unname(cs$statistic), chisq_p = cs$p.value)
  }

unif_specs <- list(

  threat_intelligence_score = c(0.5, 1), hardware_risk_avg = c(0.5, 1),

  software_risk_avg = c(0.5, 1), network_segment_risk = c(0.5, 1),

  data_classification_risk = c(0.5, 1), controls_score = c(0.5, 1),

  preventive = c(0.5, 1), detective = c(0.5, 1),

  prob_occurrence = c(0, 1), # values >1 excluded (see T02)

  avg_days_since_last_similar = c(30, 364),

  revenue_per_hour = c(1000, 10000), data_value_per_gb = c(500, 2000)

)

# T04

unif_res <- do.call(rbind, lapply(names(unif_specs), function(v) {

  r <- unif_test(df[[v]], unif_specs[[v]][1], unif_specs[[v]][2])

  data.frame(variable = v, lower = unif_specs[[v]][1], upper = unif_specs[[v]][2],

    t(round(r, 4)))

}))

unif_res$uniform_not_rejected_05 <- unif_res$ks_p > 0.05 & unif_res$chisq_p > 0.05

save_tab(unif_res, "T04_uniformity_tests")

print(unif_res)


# F02 - Empirical CDF vs theoretical uniform CDF for the target variable

x <- sort(df$threat_intelligence_score)

p2 <- ggplot(data.frame(x = x, ecdf = seq_along(x) / length(x)), aes(x)) +

  geom_step(aes(y = ecdf, colour = "Empirical CDF"), linewidth = .6) +

  geom_line(aes(y = punif(x, 0.5, 1), colour = "Uniform(0.5, 1) CDF"),

    linetype = "dashed", linewidth = .6) +

  scale_colour_manual(values = c("steelblue", "firebrick"), name = NULL) +

  labs(title = "Empirical vs theoretical uniform CDF: threat_intelligence_score",

    x = "threat_intelligence_score", y = "Cumulative probability")

save_fig(p2, "F02_ecdf_target")

```

```

# ---- 5. CORRELATION ANALYSIS -----

cor_vars <- setdiff(NUM_VARS, const_cols)          # drop zero-variance columns

cm <- cor(df[cor_vars], use = "pairwise.complete.obs")

# T05 - full matrix + summary of off-diagonal magnitudes

save_tab(data.frame(variable = rownames(cm), round(cm, 4)), "T05_correlation_matrix")

off <- cm[upper.tri(cm)]

cor_summary <- data.frame(

  n_pairs = length(off), max_abs_r = round(max(abs(off)), 4),

  mean_abs_r = round(mean(abs(off)), 4),

  n_pairs_abs_r_gt_0.05 = sum(abs(off) > 0.05),

  target_max_abs_r = round(max(abs(cm["threat_intelligence_score",

                                     setdiff(cor_vars, "threat_intelligence_score")])),

4))

save_tab(cor_summary, "T05b_correlation_summary")

print(cor_summary)


# F03 - Heatmap (colour scale fixed at [-1,1] so near-zero values read as grey)

png("outputs/figures/F03_correlation_heatmap.png", width = 2000, height = 1800, res =
300)

corrplot(cm, method = "color", type = "upper", addCoef.col = "black",

  number.cex = .45, tl.cex = .6, tl.col = "black", col.lim = c(-1, 1),

  title = "Pearson correlation matrix (all numeric variables, n = 50,000)",

  mar = c(0, 0, 2, 0))

dev.off()


# ---- 6. INTERNAL CONSISTENCY -----

# The composite existing_controls_score is nested ABOVE preventive/detective in

# the JSON. If genuine, it must be a function of them. Test that claim.

ic_mod <- lm(controls_score ~ preventive + detective, data = df)

ic_sum <- summary(ic_mod)

# T06

```

```

ic_tab <- data.frame(

  term      = rownames(ic_sum$coefficients),

  estimate  = round(ic_sum$coefficients[, 1], 5),

  std_error = round(ic_sum$coefficients[, 2], 5),

  t_value   = round(ic_sum$coefficients[, 3], 3),

  p_value   = round(ic_sum$coefficients[, 4], 4), row.names = NULL)

ic_tab <- rbind(ic_tab, data.frame(term = c("R-squared", "Adj R-squared", "F p-value"),

                                estimate = round(c(ic_sum$r.squared,

ic_sum$adj.r.squared,

                                pf(ic_sum$fstatistic[1],

ic_sum$fstatistic[2], ic_sum$fstatistic[3], lower.tail = FALSE)), 6),

                                std_error = NA, t_value = NA, p_value = NA))

save_tab(ic_tab, "T06_internal_consistency")

print(ic_tab)


# F04 - Composite vs components (sample 5,000 pts for legibility)

ic_long <- df %>% slice_sample(n = 5000) %>%

  select(controls_score, preventive, detective) %>%

  pivot_longer(-controls_score, names_to = "component", values_to = "component_score")

p4 <- ggplot(ic_long, aes(component_score, controls_score)) +

  geom_point(alpha = .15, size = .6, colour = "steelblue") +

  geom_smooth(method = "lm", colour = "firebrick", se = TRUE, linewidth = .8) +

  facet_wrap(~ component) +

  labs(title = "Composite controls score vs its stated components",

        subtitle = sprintf("OLS R-squared = %.5f | no coefficient significant at 5%%",

ic_sum$r.squared),

        x = "Component score", y = "existing_controls_score")

save_fig(p4, "F04_internal_consistency")


# ---- 7. CATEGORICAL ASSOCIATION -----

# One-way ANOVA + eta-squared: does ANY categorical variable explain variance

# in the target? (Complements the numeric-only correlation matrix.)

```

```

# T07

cat_res <- do.call(rbind, lapply(CAT_VARS, function(v) {

  a <- anova(lm(threat_intelligence_score ~ df[[v]], data = df))

  data.frame(variable = v, levels = nlevels(df[[v]]),

             F = round(a$`F value`[1], 3), p_value = round(a$`Pr(>F)`[1], 4),

             eta_squared = round(a$`Sum Sq`[1] / sum(a$`Sum Sq`), 6))

}))

save_tab(cat_res, "T07_categorical_association")

print(cat_res)


# F05 - Target by threat_type and attack_vector

p5a <- ggplot(df, aes(threat_type, threat_intelligence_score)) +

  geom_boxplot(fill = "steelblue", alpha = .6, outlier.size = .3) +

  coord_flip() + labs(title = "Threat score by threat type", x = NULL, y = NULL)

p5b <- ggplot(df, aes(attack_vector, threat_intelligence_score)) +

  geom_boxplot(fill = "darkorange", alpha = .6, outlier.size = .3) +

  coord_flip() + labs(title = "Threat score by attack vector", x = NULL, y = NULL)

save_fig(p5a, "F05a_target_by_threat_type", h = 3.6)

save_fig(p5b, "F05b_target_by_attack_vector", h = 3.2)


# ---- 8. PREDICTIVE MODELLING (5-fold CV, 4 targets x 4 models) -----

FEATURES <- c(CAT_VARS[CAT_VARS != "scenario_type"],

              setdiff(NUM_VARS, c(TARGETS, const_cols)))

# Target leakage verification - run once

cat("\nTarget leakage check:\n")

for (target_check in TARGETS) {

  cat(target_check, "present in FEATURES:", target_check %in% FEATURES, "\n")

}

samp <- df[sample(nrow(df), 10000), ]

```

```
K <- 5
```

```
fit_predict <- function(model, train, test) {  
  
  switch(model,  
  
    "Linear Regression" = predict(lm(y ~ ., train), test),  
  
    "Decision Tree"      = predict(rpart(y ~ ., train, method = "anova"), test),  
  
    "Random Forest"      = predict(randomForest(y ~ ., train, ntree = 100, nodesize  
= 20,  
  
                                              maxnodes = 200), test),  
  
    "XGBoost" = {  
  
      mm_tr <- model.matrix(y ~ . - 1, train)  
  
      mm_te <- model.matrix(y ~ . - 1, test)  
  
      bst <- xgboost(data = mm_tr, label = train$y, nrounds = 200,  
  
                    max_depth = 4, eta = 0.05, subsample = .8,  
  
                    colsample_bytree = .8,  
  
                    objective = "reg:squarederror", verbose = 0)  
  
      predict(bst, mm_te)  
  
    })  
  
  }  
}
```

```
MODELS <- c("Linear Regression", "Decision Tree", "Random Forest", "XGBoost")
```

```
cv_results <- list()
```

```
for (tg in TARGETS) {  
  
  preds <- setdiff(FEATURES, tg)  
  
  
  d <- samp[, c(preds, tg)]  
  
  d <- d[complete.cases(d), ]  
  
  names(d)[ncol(d)] <- "y"  
  
  
  folds <- sample(rep(1:K, length.out = nrow(d)))
```

```

for (m in MODELS) {

  r2s <- rmsees <- numeric(K)

  for (k in 1:K) {

    tr <- d[folds != k, ]

    te <- d[folds == k, ]

    p <- fit_predict(m, tr, te)

    r2s[k] <- r2(te$y, p)

    rmsees[k] <- rmse(te$y, p)

  }

  base_rmse <- mean(sapply(1:K, function(k)

    rmse(d$y[folds == k], mean(d$y[folds != k])))

  ))

  cv_results[[length(cv_results) + 1]] <- data.frame(

    target = tg,

    model = m,

    cv_r2_mean = round(mean(r2s), 4),

    cv_r2_sd = round(sd(r2s), 4),

    cv_rmse_mean = round(mean(rmsees), 5),

    baseline_rmse_mean = round(base_rmse, 5),

    rmse_vs_baseline_pct =

      round(100 * (mean(rmsees) - base_rmse) / base_rmse, 2)

  )

  cat(sprintf("%-26s %-18s CV R2 = %7.4f (sd %4.4f)\n",

    tg, m, mean(r2s), sd(r2s)))

}

```

```

}

cv_tab <- do.call(rbind, cv_results)

save_tab(cv_tab, "T08_cv_model_results")

# ---- 9. FEATURE IMPORTANCE (primary target, full model) -----

d0 <- samp[, c(setdiff(FEATURES, "threat_intelligence_score"),
"threat_intelligence_score")]

d0 <- d0[complete.cases(d0), ]; names(d0)[ncol(d0)] <- "y"

rf0 <- randomForest(y ~ ., d0, ntree = 200, nodesize = 20, maxnodes = 200, importance =
TRUE)

imp <- as.data.frame(importance(rf0)); imp$feature <- rownames(imp)

imp <- imp %>% arrange(desc(`%IncMSE`)) %>% mutate(across(where(is.numeric), ~ round(.x,
4)))

save_tab(imp, "T09_rf_feature_importance") # T09

p7 <- ggplot(imp, aes(reorder(feature, `%IncMSE`), `%IncMSE`)) +

  geom_col(fill = "steelblue") + coord_flip() +

  labs(title = "Random Forest permutation importance (%IncMSE), target =
threat_intelligence_score",

        subtitle = "Values near/below zero indicate no useful predictive contribution",

        x = NULL, y = "% increase in MSE when permuted")

save_fig(p7, "F07_rf_feature_importance", h = 4.8)

# ---- 10. PERMUTATION (LABEL-SHUFFLE) TEST -----

# If the model performs the same when the target is randomly shuffled, the

# "real" model has learned nothing beyond noise.

perm_test <- function(d, n_perm = 10) {

  idx <- sample(nrow(d), round(.8 * nrow(d))); tr <- d[idx, ]; te <- d[-idx, ]

  real <- r2(te$y, predict(randomForest(y ~ ., tr, ntree = 100, nodesize = 20, maxnodes
= 200), te))

  perms <- replicate(n_perm, {

    trs <- tr; trs$y <- sample(trs$y)

```

```

      r2(te$y, predict(randomForest(y ~ ., trs, ntree = 100, nodesize = 20, maxnodes =
200), te))

    })

    c(real_r2 = real, perm_mean_r2 = mean(perms), perm_sd_r2 = sd(perms),

      p_value = mean(perms >= real))

  }

pt <- round(perm_test(d0), 4)

perm_tab <- data.frame(target = "threat_intelligence_score", t(pt))

save_tab(perm_tab, "T10_permutation_test")    # T10

print(perm_tab)


# F08

p8 <- ggplot(data.frame(condition = c("Real target", "Shuffled target (mean of 10)"),

      r2 = c(pt["real_r2"], pt["perm_mean_r2"])),

  aes(condition, r2, fill = condition)) +

  geom_col(width = .5) + geom_hline(yintercept = 0, linetype = "dashed") +

  scale_fill_manual(values = c("steelblue", "grey60")) + theme(legend.position = "none")
+

  labs(title = "Permutation test: Random Forest R-squared, real vs shuffled target",

    subtitle = sprintf("Permutation p-value = %.2f", pt["p_value"]), x = NULL, y =
"Test R-squared")

save_fig(p8, "F08_permutation_test", h = 3.6)


# ---- 11. LEARNING CURVE -----

# Does more data help? For a genuine signal, R2 rises with n. For noise, it stays ~0.

lc <- do.call(rbind, lapply(c(500, 1000, 2500, 5000, 10000), function(n) {

  dd <- d0[sample(nrow(d0), min(n, nrow(d0))), ]

  idx <- sample(nrow(dd), round(.8 * nrow(dd)))

  m <- randomForest(y ~ ., dd[idx, ], ntree = 100, nodesize = 20, maxnodes = 200)

  data.frame(n_train = length(idx), test_r2 = round(r2(dd$y[-idx], predict(m, dd[-idx,
])), 4))

}))

save_tab(lc, "T11_learning_curve")    # T11

```

```

p9 <- ggplot(lc, aes(n_train, test_r2)) + geom_line(colour = "steelblue") +

  geom_point(size = 2, colour = "steelblue") + geom_hline(yintercept = 0, linetype =
"dashed") +

  scale_x_continuous(labels = comma) +

  labs(title = "Learning curve: Random Forest test R-squared vs training size",

        x = "Training observations", y = "Test R-squared")

save_fig(p9, "F09_learning_curve", h = 3.6)


# ---- 12. CLASSIFICATION RE-FRAMING TEST -----

# Supervisor option 2: would binning severity into Low/Medium/High help?

d0$y_class <- cut(d0$y, breaks = quantile(d0$y, c(0, 1/3, 2/3, 1)),

                  labels = c("Low", "Medium", "High"), include.lowest = TRUE)

idx <- sample(nrow(d0), round(.8 * nrow(d0)))

rfc <- randomForest(y_class ~ . - y, d0[idx, ], ntree = 100, nodesize = 20, maxnodes =
200)

pred_c <- predict(rfc, d0[-idx, ])

acc <- mean(pred_c == d0$y_class[-idx])

majority <- max(prop.table(table(d0$y_class[-idx])))

cls_tab <- data.frame(metric = c("RF 3-class accuracy", "Majority-class baseline",

                                "Chance (3 balanced classes)"),

                      value = round(c(acc, majority, 1/3), 4))

save_tab(cls_tab, "T12_classification_reframing") # T12

print(cls_tab)

print(table(predicted = pred_c, actual = d0$y_class[-idx]))


# ---- 13. SESSION INFO (reproducibility, appendix of Technical Report) -----

writeLines(capture.output(sessionInfo()), "outputs/session_info.txt")

cat("\nDONE. Tables in outputs/tables, figures in outputs/figures.\n")

```

Appendix 2 — R Session Information

R version 4.5.1 (2025-06-13 ucrt)

Platform: x86_64-w64-mingw32/x64

Running under: Windows 11 x64 (build 26200)

Matrix products: default

LAPACK version 3.12.1

locale:

*[1] LC_COLLATE=English_India.utf8 LC_CTYPE=English_India.utf8
LC_MONETARY=English_India.utf8*

[4] LC_NUMERIC=C LC_TIME=English_India.utf8

time zone: Europe/London

tzcode source: internal

attached base packages:

[1] stats graphics grDevices utils datasets methods base

other attached packages:

[1] scales_1.4.0 corrplot_0.95 xgboost_3.2.1.1 rpart_4.1.24

[5] randomForest_4.7-1.2 ggplot2_4.0.3 tidyr_1.3.1 dplyr_1.2.0

[9] stringr_1.5.2

loaded via a namespace (and not attached):

*[1] utf8_1.2.6 generics_0.1.4 stringi_1.8.7 lattice_0.22-7
digest_0.6.37*

*[6] magrittr_2.0.4 evaluate_1.0.5 grid_4.5.1 RColorBrewer_1.1-3
fastmap_1.2.0*

*[11] jsonlite_2.0.0 Matrix_1.7-3 mgcv_1.9-3 purrr_1.1.0
textshaping_1.0.3*

*[16] cli_3.6.5 rlang_1.1.7 splines_4.5.1 withr_3.0.2
yaml_2.3.10*

*[21] parallel_4.5.1 tools_4.5.1 vctrs_0.7.1 R6_2.6.1
lifecycle_1.0.5*

```
[26] ragg_1.5.0          pkgconfig_2.0.3      pillar_1.11.1        gtable_0.3.6
glue_1.8.0

[31] data.table_1.17.8    systemfonts_1.3.1    xfun_0.53             tibble_3.3.0
tidyselect_1.2.1

[36] rstudioapi_0.17.1    knitr_1.50           farver_2.1.2          htmltools_0.5.8.1
nlme_3.1-168

[41] rmarkdown_2.30       labeling_0.4.3        compiler_4.5.1        S7_0.2.0
```

Appendix 3 — Solution Screenshots

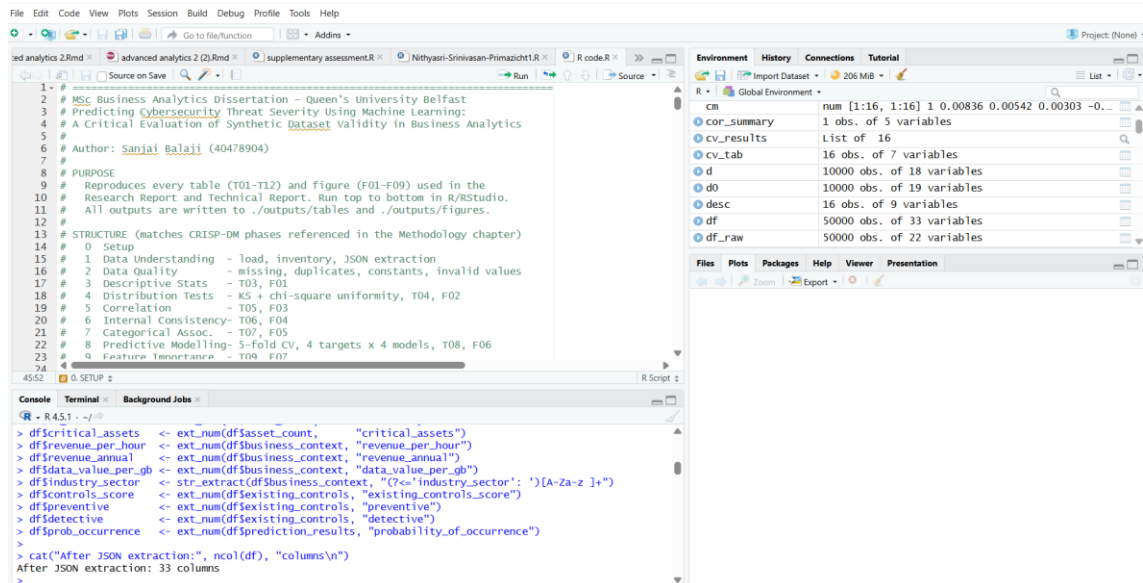
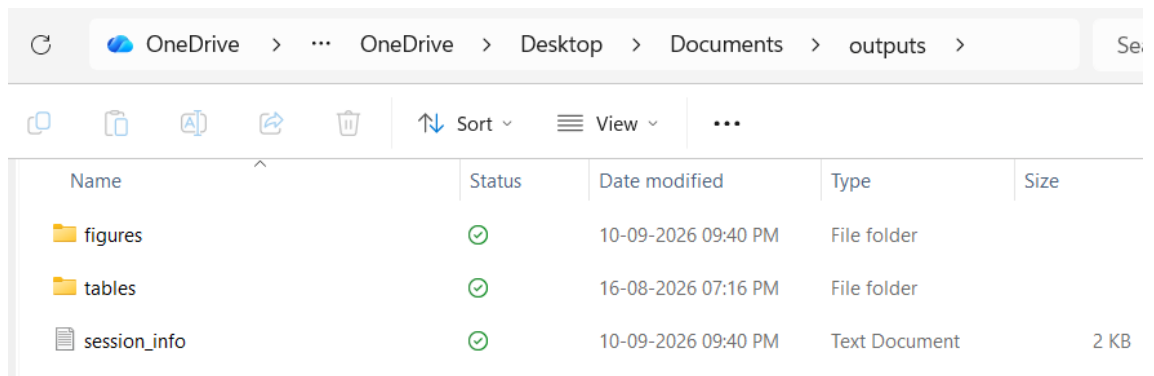


Figure 1-R Studio environment showing the loaded dataset and the 33 columns after JSON extraction

	target	model	cv_r2_mean	cv_r2_sd	cv_rmse_mean	baseline_rmse_mean	rmse_vs_baseline_pct
1	threat_intelligence_score	Linear Regression	-0.0039	0.0035	0.14542	0.14516	0.18
2	threat_intelligence_score	Decision Tree	-0.0003	0.0002	0.14516	0.14516	0.00
3	threat_intelligence_score	Random Forest	-0.0064	0.0005	0.14560	0.14516	0.31
4	threat_intelligence_score	XGBoost	-0.0184	0.0067	0.14646	0.14516	0.90
5	prob_occurrence	Linear Regression	-0.0027	0.0031	0.29004	0.28966	0.13
6	prob_occurrence	Decision Tree	-0.0001	0.0001	0.28966	0.28966	0.00
7	prob_occurrence	Random Forest	-0.0040	0.0025	0.29022	0.28966	0.19
8	prob_occurrence	XGBoost	-0.0179	0.0035	0.29223	0.28966	0.88
9	controls_score	Linear Regression	-0.0036	0.0020	0.14511	0.14487	0.17
10	controls_score	Decision Tree	-0.0002	0.0003	0.14487	0.14487	0.00
11	controls_score	Random Forest	-0.0081	0.0026	0.14544	0.14487	0.39
12	controls_score	XGBoost	-0.0218	0.0056	0.14642	0.14487	1.07
13	critical_assets	Linear Regression	-0.0049	0.0026	1.11351	1.11158	0.17
14	critical_assets	Decision Tree	-0.0014	0.0014	1.11158	1.11158	0.00
15	critical_assets	Random Forest	-0.0074	0.0039	1.11492	1.11158	0.30
16	critical_assets	XGBoost	-0.0184	0.0027	1.12098	1.11158	0.85

Showing 1 to 16 of 16 entries. 7 total columns

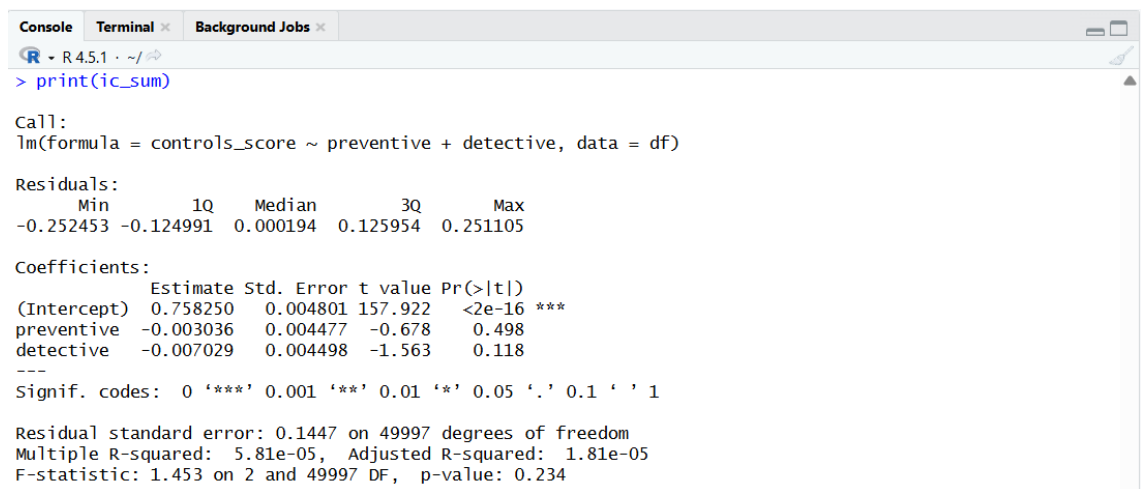
Figure 2- Console output of the sixteen cross-validated model results



The screenshot shows a OneDrive file explorer window with the path: OneDrive > Desktop > Documents > outputs. The directory contains three items: a folder named 'figures', a folder named 'tables', and a text document named 'session_info' which is 2 KB in size. All items have a green checkmark status icon.

Name	Status	Date modified	Type	Size
figures	✓	10-09-2026 09:40 PM	File folder	
tables	✓	16-08-2026 07:16 PM	File folder	
session_info	✓	10-09-2026 09:40 PM	Text Document	2 KB

Figure 3-The outputs directory structure showing tables and figures folders



The screenshot shows the R console output for the command `print(ic_sum)`. It displays the call to `lm()`, the residuals, coefficients, and model fit statistics.

```

> print(ic_sum)

Call:
lm(formula = controls_score ~ preventive + detective, data = df)

Residuals:
    Min       1Q   Median       3Q      Max
-0.252453 -0.124991  0.000194  0.125954  0.251105

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.758250   0.004801  157.922  <2e-16 ***
preventive   -0.003036   0.004477  -0.678   0.498
detective    -0.007029   0.004498  -1.563   0.118
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1447 on 49997 degrees of freedom
Multiple R-squared:  5.81e-05, Adjusted R-squared:  1.81e-05
F-statistic: 1.453 on 2 and 49997 DF, p-value: 0.234

```

Figure 4-Console output of the internal consistency regression summary

```

> # Target leakage verification – run once
> cat("\nTarget leakage check:\n")

Target leakage check:
> for (target_check in TARGETS) {
+   cat(target_check, "present in FEATURES:", target_check %in% FEATURES, "\n")
+ }
threat_intelligence_score present in FEATURES: FALSE
prob_occurrence present in FEATURES: FALSE
controls_score present in FEATURES: FALSE
critical_assets present in FEATURES: FALSE

```

Figure 5-Console output showing the corrected target leakage result